Student 9 **GANESHRAM.M** 192321060RD 5 / 5 submitted

Q1. ATM Withdrawal – Exception Handling Debugging

CODE

```
import java.util.*;
public class ATM{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);

        try{
            if(!sc.hasNextInt()){
                System.out.println("Invalid input");
                return;
            }
            int balance=sc.nextInt();
            if(!sc.hasNextInt()){
                System.out.println("Invalid input");
                return;
            }
            int amount=sc.nextInt();
            if(amount<=0){
                System.out.println("Invalid Withdrawal amount:");
            }
            else if(amount>balance){
                System.out.println("Insufficient balance");
            }
            else{
                System.out.println("Withdrawal sucessfull");
                System.out.println("Remaining balance = " +(balance-amount));
            }

        }catch(InputMismatchException e){
            System.out.println("Invalid input:");
        }catch(ArithmeticException e){
            System.out.println("Transaction failed");
        } finally{
            System.out.println("Transaction completed");
        }
    }
}
```

OUTPUT

```
Invalid input
Transaction completed
```

Q2. Hospital Registration – NullPointerException Debugging

CODE

```
import java.util.Scanner;
import java.util.InputMismatchException;
class patientRegistration{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            System.out.println("Enter a patient name:");
            String inputName=sc.nextLine();
            String name;
            if(inputName.equalsIgnoreCase("Null")|| inputName.trim().isEmpty()){
                name="unknown";
            }
            else{
                name = inputName;
            }
            System.out.println("Enter age:");
            try{
                int age=sc.nextInt();
                System.out.println("patient Name:"+name.toUpperCase());
                if(age>=18){
                    System.out.println("Adult patient");
                }else{
                    System.out.println("Minor patient");
                }
            }
            catch(InputMismatchException e){
                System.out.println("Invalid age.");
            }
        }catch(Exception e){
            System.out.println("Error occured during registration");
        }
        finally{
            System.out.println("Registation completed");
        }
    }
}
```

INPUT

```
Null
25
```

OUTPUT

```
Enter a patient name:
Enter age:
patient Name:UNKNOWN
Adult patient
Registation completed
```

Q3. E-Commerce Product Search – Array Exception Debugging

CODE

```
import java.util.Scanner;
import java.util.InputMismatchException;
class ProductSearch{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] products ={"Laptop","Mobile","Tablet","Headphone","Keyboard"};
        try{
            System.out.println("Enter a product number (1-5):");
            int productNumber = sc.nextInt();
            if(productNumber<1||productNumber>5){
                System.out.println("Invalid product number");
            }else{
                System.out.println("selected product:" + products[productNumber-1]);
            }
        }catch(InputMismatchException e){
            System.out.println("Invalid input");
        }finally{
            System.out.println("search completed");
        }
    }

}
```

INPUT

```
4
```

OUTPUT

```
Enter a product number (1-5):
selected product:Headphone
search completed
```

Q4. Railway Reservation – Synchronization Debugging

CODE

```
class RailwayReservation{
    public static void main(String[] args){
        Reservation reservation=new Reservation();

        Customer c1=new Customer(reservation,"Customer 1");
        Customer c2=new Customer(reservation,"Customer 2");

        c1.start();
        c2.start();
    }
}
class Reservation{
    int seats =1;
    synchronized void bookSeat(String customer){
        if(seats>0){
            System.out.println(customer +" is booking a seat");
            try{
                Thread.sleep(100);
            }
            catch(InterruptedException e){
                System.out.println("Thread interruped");
            }
            seats--;
            System.out.println(customer +" - booking succesfull");
        }else{
            System.out.println(customer +" No seat available");
        }
    }
}
class Customer extends Thread{
    Reservation reservation;
    String customerName;

    Customer(Reservation reservation,String customerName){
        this.reservation=reservation;
        this.customerName=customerName;
    }
    public void run(){
        reservation.bookSeat(customerName);
    }
}
```

OUTPUT

```
Customer 1 is booking a seat
Customer 1 - booking succesfull
Customer 2 No seat available
```

Q5. Bank Account – Race Condition Debugging

CODE

```
class BankApplication{
    public static void main(String[] args) throws InterruptedException{
        BankAccount account = new BankAccount();
        ATM atm1 = new ATM(account,7000);
        ATM atm2 = new ATM(account,6000);

        atm1.setName("ATM-1");
        atm2.setName("ATM-2");

        atm1.start();
        atm2.start();

        atm1.join();
        atm2.join();
        System.out.println("Final Balance = " + account.balance);
    }
}
class BankAccount {
    int balance = 10000;
    synchronized void withdraw(int amount)
    {
        if (balance >= amount){
            System.out.println(Thread.currentThread().getName()+" Withdrawing"+ amount);
       try{
           Thread.sleep(100);
       }catch(InterruptedException e){
           System.out.println("Interrupted");
       }
       balance = balance-amount;
       System.out.println(Thread.currentThread().getName()+" Withdrawal succesful");
   } else{
       System.out.println( Thread.currentThread().getName()+" insufficient balance");
        }
    }
}
class ATM extends Thread {
    BankAccount account;
    int amount;
    ATM(BankAccount account, int amount){
        this.account = account;
        this.amount = amount;
    }
    public void run (){
        account.withdraw(amount);
    }
}
```

OUTPUT

```
ATM-1 Withdrawing7000
ATM-1 Withdrawal succesful
ATM-2 insufficient balance
Final Balance = 3000
```